

早起的鸟儿有虫吃：更优的早期生命周期缺陷预测器

N. C. 施里坎特和蒂姆·门齐斯

摘要 —— 在研究人员急于对所有可用数据进行推理之前，他们应该首先检查信息是否在某个小区域内最为密集。我们之所以这么说，是因为在 240 个 GitHub 项目中，我们发现数据中的信息“聚集”在项目的早期阶段。事实上，仅从最初的 150 次提交中学习到的缺陷预测模型，其效果与最先进的模型相当，甚至更好。仅使用这些早期生命周期数据，我们就可以非常快速地构建模型（使用几周的 CPU 时间，而不是几个月）。此外，我们可以找到简单的模型（仅包含两个特征），这些模型可以推广到数百个软件项目。

基于这一经验，我们提醒，先前关于推广软件工程缺陷预测模型的研究可能不必要地将一个本质上简单的过程复杂化了。此外，先前专注于软件生命周期后期数据的研究现在需要重新审视，因为它们是从相对缺乏信息的区域得出的。

复现说明：我们所有的数据和脚本都可在<https://github.com/snaraya7/early-defect-prediction-tse>上获取。

关键词 — 缺陷预测、软件分析、迁移学习

arXiv:
2105.11082v1
[计算机科学].
软件工
程] 2021
年 5 月
24 日

1 引言

在缺陷预测中，研究人员通常“渴求数据”，并假定如果数据有用，那么更多的数据就更加有用。例如：

- “长期即时编译（JIT）模型应该使用大量变更的缓存来进行训练” [1]；
- “……只要规模足够大；（预测性能）很可能会因样本规模而得到更大提升” [2]；
- “人们很自然地认为，积累多个版本可能是有益的，因为它代表了一个项目的可变性” [3]。

在我们的工作中，我们有时会看到这种数据密集型方法产生的结果不尽人意。例如，有一次我们尝试从 70 多万次代码提交中尽可能地学习。这个过程所需的网络数据提取工作，使用五台 16 核机器，耗费了近 500 天的 CPU 时间，历时 7 天（因为这些项目的数据必须进行清理、处理并转换为某种标准格式）。在这个数据空间内，我们发现从数据的不同部分学习到的模型存在显著差异。所以，即便做了这么多工作，耗费了一年多的 CPU 时间，我们还是无法向业务用户报告一个稳定的结论。

如果“过多的数据”可能会适得其反，那么究竟多少数据才足以构建有效的缺陷预测模型呢？我们对这个问题的答案源于图 1 中展示的一种此前未被报道的效应。如图所示，当我们观察 GitHub 项目中有缺陷提交的占比时，一个显著的模式就会显现出来。具体而言，大多数有缺陷的提交发生在生命周期的早期阶段。

这一观察结果促使对“轻数据”方法展开研究，该方法仅使用早期生命周期数据来预测缺陷。3.4 节的文献综述表明，

令人惊讶的是，这种缺陷预测方法在之前的研究中被忽视了。这是一个重大疏忽，因为在 240 个 GitHub 项目中，我们可以证明，从最初 150 次提交中学习到的缺陷预测器，其效果与最先进的的方法相当，甚至更好。

总体而言，本文的贡献在于表明：

- 项目中的信息在其生命周期内可能分布不均。对于此类数据，采用“轻量级数据”方法可能非常有用。
- 例如，通过使用早期生命周期数据，我们发现了简单的模型（仅包含两个特征），这些模型可在数百个项目通用。与传统方法相比，此类模型的构建速度要快得多（构建时间从数月的 CPU 时间缩短至数周）。
- 因此，在研究人员使用所有可用数据之前，他们首先需要检查其（例如）错误提交数据在整个生命周期中出现的频率是否相同。我们这样说是因为之前许多关于从多个项目中学习的方法的研究 [4]–[53] 不必要地将一个本质上简单的过程复杂化了。

本文其余部分将阐述我们的方法，并提供实验证据，以证明这种早期生命周期迁移学习算法比其他迁移学习方法更简单、更快速且更稳定。在此之前，我们先岔开话题，回应一下针对我们结论的明显质疑。此处展示的结果仅适用于缺陷预测。在未来的研究中，我们需要测试这些结果是否适用于其他领域。

2 与先前工作的联系

此前，我们已在 ICSE' 21 [54] 1 报告了图 1.A 的结果。经我们计算，本文只有图 1.A 和 2.5 页内容（如 § 6.1）来自该前期工作。

对于评审人员，我们请注意该论文可在
<https://arxiv.org/pdf/2011.13071.pdf>在线获取。

• 作者们来自美国罗利市北卡罗来纳州立大学计算机科学系。邮箱：
snaraya7@ncsu.edu, timmm@ieee.org

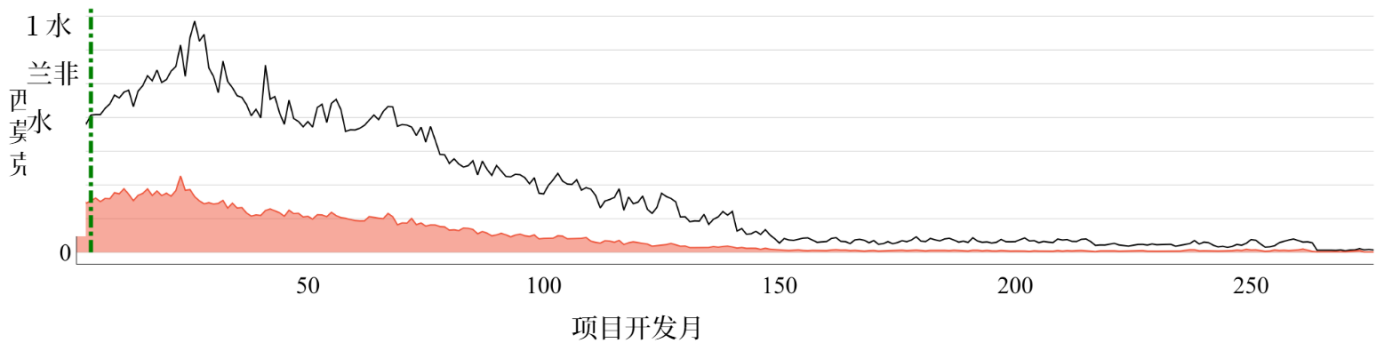


图 1.A: 155 个热门 GitHub 项目 (星标数 > 1000)。数据来自 120 万次提交。

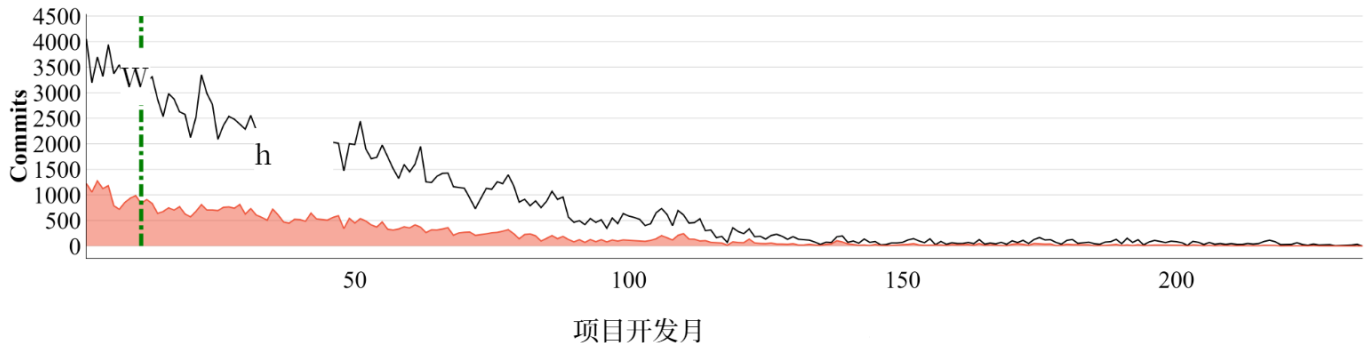


图 1.B: 85 个冷门的 GitHub 项目 ($\#stars < 1000$)。数据来源于 253,289 次提交。

图 1: 大多数有缺陷的提交发生在生命周期的早期。黑色: 红色 = 无缺陷提交: 有缺陷提交。在本文中, 我们将 (a) 截至绿色垂直 (虚线) 线所学习到的模型与 (b) 使用更多数据所学习到的模型进行比较。

至于本文与先前工作在语义上的差异, 先前的研究更为局限, 因为它没有图 1.B 的额外数据。此外, 先前的研究仅对图 1.A 的数据运行了一些项目内采样方法。本文更进一步, 将我们的方法与迁移学习方法进行了比较。再者, 此前我们并未报告通过这种方法学到的模型。我们在此表明, 一个非常简单的模型 (仅使用几个变量) 可以从几个早期生命周期样本中学习得到。最后, 正如本文表 4 中的新结果所示, 我们能够超越先前工作的结果。

3 背景

3.1 关于缺陷预测

本文的案例研究基于缺陷预测。因此, 在进行其他任何操作之前, 我们需要先介绍一下该研究领域。

修复软件缺陷成本不菲 [55]。因此, 数十年来, 软件工程研究人员一直在设计各种方法, 以便在部署前预测软件质量。最早的研究之一是 1971 年秋山 (Akiyama) 针对日本富士通开发的一个系统, 使用基于规模的缺陷预测方法开展的研究 [56]。这种方法至今仍很流行。2018 年对来自 33 个国家和五大洲的 395 名从业者进行的一项调查 [20] 发现, 超过 90% 的受访者愿意采用缺陷预测技术 [57]。

缺陷预测使用数据挖掘工具输入静态代码属性, 并输出预测代码中最可能存在漏洞位置的模型 [58], [59]。以这种方式学习得到的模型非常有效, 而且构建成本相对较低:

- 实际上: 米西里利等人 [60] 以及金等人 [61] 报告称, 在指导工业质量保证流程中使用此类预测指标时, 可节省大量成本。
- 实现起来相对简单。此外, 拉赫曼等人 [62] 表明, 此类预测器与更为复杂的方法相比具有竞争力。例如, 他们指出静态代码分析工具的许可证可能很昂贵, 并且在任何重大语言升级后都需要更新。另一方面, 缺陷预测器可以通过对项目数据进行一些轻量级解析来快速实现。

3.2 迁移学习

缺陷预测模型是通过项目数据学习得到的。如果没有足够的数据来学习这些模型, 会发生什么情况呢? 对于较新的项目 (以及一些遗留项目缺乏历史数据的情况 [63]), 这是一个尤为严峻的问题。

在这种情况下, 从业者和研究人员可能会找出与本地项目有一些相似之处的成熟项目。一旦找到, 便可将从旧项目中吸取的经验应用到新项目中。转移的方式有多种:

- CPDP: 跨项目缺陷预测。从其他项目中吸取的经验教训, 并应用于本项目。
- WPDP: 项目内缺陷预测。利用来自该项目的数据, 借鉴以往经验教训, 对项目生命周期后期的开发情况进行预测。

至少可以说，迁移学习是软件工程（SE）中一个非常活跃的研究领域。仅在过去五年里，我们就可以找到 1000 多篇相关文章（通过）

查询“跨项目缺陷预测”，2)。据我们统计，

在该语料库中，至少有二十多种迁移学习方法 [3]。该研究中出现的一些有趣方法包括：

- 异构迁移，允许以不同格式表示的数据在项目之间转移 [40]；
- 时间迁移学习是一种项目内缺陷预测（WPDP）工具，它利用早期生命周期数据对生命周期后期进行预测 [64]。

本文使用相同的工具从全部 240 个项目中收集数据。此外，回顾图 1 中的垂直绿线，我们正在使用先前的项目数据对未来进行预测。因此，本文探讨的是一种非异构时间迁移学习方法。

我们对相关文献的研读表明，除了我们自己的研究，软件工程（SE）文献中此前最先进的方法是 Nam 等人提出的 TCA + 方法 [65]。给定来自某些源项目和目标项目的数据，TCA 旨在通过维度旋转、扩展和收缩来“对齐”源数据和目标数据。TCA + 是对基本 TCA 的扩展，它使用自动化方法为 TCA 寻找归一化选项。

3.3 数据密集型与轻数据型

我们对文献的解读是，最近的软件演化（SE）分析论文采用了一种数据密集型方法（例如，见我们引言中来自 [1]-[3] 的引用）。然而，仅仅因为一种技术受欢迎并不意味着就应该推荐它。我们认为，在采用数据密集型方法之前尝试数据精简型方法可能会有所帮助。一方面，所有这些数据可能无法获取。在工业环境中，更多数据的可用性并不能得到保证。此外，从更多数据中学习可能并无益处。从业者可能无法轻易获取专有数据，以便为他们的本地项目构建预测模型。齐默尔曼等人表明，从同一领域迁移预测模型并不能保证预测质量 [66]。最后，出于隐私考虑，即使是同一组织内的团队，也可能不会轻易将其成熟项目提供给他们团队使用 [15]。

另一方面，一些研究人员报告称，数据密集型方法可能存在问题：

- 在引言中，我们阐述了从 70 多万条 GitHub 提交记录中学习时发现的问题。
- 在 2011 年软件工程教育与培训会议（ESEM' 11）的主题演讲中，伊莱恩·韦尤克（Elaine Weyuker）质疑自己是否曾有机会公开美国电话电报公司（AT&T）的信息 [67]。
- 在 30 多年使用 COCOMO 的过程中，Boehm 只能分享 200 个项目的成本估算数据。

3.4 相关工作

探索早期生命周期数据推理的一个原因是，此前从未有人做过这项工作。在软件工程（SE）迁移学习的大部分研究中，

2. 2020 年在 <https://scholar.google.co.in/> 上进行了查询

表 1: 2010 年以来开展一项或多项因果预测深度学习过程（CPDP）实验的 50 篇“高被引”（每年被引用超 10 次）论文。

Year	#Data	#Features	Projects	Cites/Year	Paper
2011		20	2	16.44	[4]
		198	6	19.25	[5]
		17	10	39.13	[6]
2012		20	10	30.13	[7]
		20	7	23.38	[8]
		20	2	13.88	[9]
		8	9	25.38	[10]
		16	41	14	[11]
		17	8	48.43	[12]
2013		20	41	22.43	[13]
		20	10	19.57	[14]
		20	10	13.29	[15]
		20	14	13.29	[16]
		20	10	11.43	[17]
		54	12	32	[18]
2014		14	11	16.5	[19]
		20	10	20.5	[20]
		26	1398	18.5	[21]
		18	7	13.4	[22]
2015		20	11	19.8	[23]
		20	10	14.4	[24]
		20	17	12.4	[25]
		20	10	11.2	[26]
		28	11	24.4	[27]
		20	10	33	[28]
	None	X	26	35.5	[29]
		14	11	26.5	[30]
		14	6	19.75	[31]
2016		17	10	23.5	[32]
		20	10	35.5	[33]
		20	21	20.75	[34]
		20	30	11.75	[35]
		26	1390	16.25	[36]
		61	23	10.5	[37]
		X	10	70.25	[38]
2017		20	15	21.33	[39]
		61	34	80	[40]
		61	8	10.67	[41]
		6	58	21.5	[42]
		14	6	20.5	[43]
2018		20	11	23	[44]
		61	18	18.5	[45]
		61	28	14.5	[46]
		20	10	14	[47]
		61	16	19.5	[48]
		X	10	20	[49]
2019		14	6	17	[50]
		61	8	14	[51]
		20	7	20	[52]
		20	14	19	[53]

关键：部分、整体、无 - 无训练数据，X - 自动生成的特征

方法 [11] - [15]、[19]、[20]、[22] - [27]、[30] - [37]、[39] - [47]、[50] 可被描述为“数据饥渴型”，因为它们从一个或多个项目中转移了所有可用数据来构建其预测模型。为了更深入地了解软件工程中数据驱动的推理，我们进行了文献检索，结果总结在表 1 中。2021 年 3 月，我们在谷歌学术上使用查询词（跨项目缺陷预测）找到了自 2011 年以来发表的 982 篇文章。遵循阿格拉瓦尔等人 [68] 以及马修斯等人 [69] 的建议，我们专注于“高被引”论文，即那些每年被引用超过十次的论文。阅读这些论文，并剔除纯粹的综述性质的论文（或与跨项目缺陷预测无关的论文）后，我们找到了 50 篇进行了一些迁移学习实验的论文。在这组论文中，我们发现了三种行选择方法：

- 如果该论文的方法从一个或多个项目中转移所有行，则为“整体”。
- “部分”，如果该论文的方法探索了一个或多个项目的大型搜索空间，以找到一个小的

值得传输数据的行集合。

- 在一个案例中，2016 年，我们还发现了一种“无”方法，该方法不进行训练，而是直接对测试数据进行聚类以找出异常值，然后将这些异常值标记为错误 [29]。鉴于已记录了该方法，本文将不再探讨这种小众方法。

请注意，无论是“整体”还是“部分”，该分析都会在返回部分或全部数据之前，对整个生命周期中的数据进行考察。

至于其他类型的数据选择，对于所有这些论文，我们统计了：

- 这些研究中使用的项目数量；
- 他们的预测器所使用的特征数量。

当我们试图将本文纳入表 1 时，我们发现我们的方法确实是超乎寻常的。本文所倡导的方法既非“整体”也非“局部”，因为我们从早期数据中学习，然后停止收集数据（因此与表 1 中的所有研究不同，我们从未查看过所有数据）。此外，我们的“项目数量”等于 1（这在表 1 中甚至都未出现），而且我们只使用了少数几个特征（远远少于表 1 中其他研究使用的特征）。

因此，我们有一定把握断言，本文所采用的方法此前尚未有人探索过。

3.4.1 代表性技术

为了设计一个合适的实验，我们采用了以下准则。

首先，我们将早期样本与在更大范围的项目数据空间中进行的采样进行比较。因此，在接下来的内容中，我们将展示早期实验与所有实验的对比情况。

其次，有两种查找数据的方法：项目内和跨项目。因此，我们将早期的数据精简型和所有数据密集型实验分为：

- 轻量数据：项目内早期数据和跨项目早期数据
- 数据密集型：全项目内和全跨项目

最后，查阅相关文献，我们可以看到一些在我们的研究中应该采用的前沿算法（具体来说，TCA + 和领军者跨项目学习方法 [12], [37]）。因此，在探索跨项目学习时，我们将采用这些方法。

要解释本研究中使用的具体方法，需要进一步详细介绍这些算法。

有关算法细节以及我们选择的算法的讨论，请参阅

§ 4.1.1。但是，概括地说，据我们所知，这些技术的数据精简变体在软件工程领域之前尚未被探索过。

4 实验方法

本文其余部分针对图 1 中的数据，对项目早期生命周期内的迁移学习与其他学习策略进行了实验评估。

4.1 数据

本研究中使用的所有数据均来自托管在 GitHub 上的开源项目，其他软件工程研究人员可以复现这些数据。有关该数据的详细信息，请参阅表 2、图 2 和图 3。

为了挖掘这些数据，我们研究了 Munaiah 等人 [72] 整理的数据集，该数据集有许多标准来区分工程项目和普通项目。然后，我们使用 Commit-Guru [71] 挖掘了所有项目，方法与这项工作 [54] 类似。根据软件工程领域 GitHub 挖掘者 [54]、[73] 的标准合理性检查，部分项目被排除在外：

- 缺陷率低于 1%；
- 少于两个版本；
- 活动时间不足一年；
- 无许可证信息；
- 缺陷提交和无缺陷提交均少于 5 个。

我们进行了两次采样：

- 一次针对热门项目，即那些在 GitHub 上“星标”排名超过 1000 的项目；
- 一次针对不受欢迎的项目，即那些在 GitHub 上“星标”排名低于 1000 的项目。

这种抽样产生了 155 个热门项目和 85 个冷门项目。由于穆纳亚等人 [72] 使用了 1000 颗星作为划分标准，因此我们也采用这一标准。此外，观察图 2，我们发现 1000 颗星上下的分布情况差异很大。具体而言，1000 颗星以上和以下的星数中位数分别为 9149 和 47。

以这种方式收集的项目是用广泛使用的编程语言（包括 Java、Python、C、C++、C#、Kotlin、JavaScript、Ruby、PHP、Fortran、Go、Shell 等）为不同领域开发的。

本研究中使用的全部 14 个特征列于表 2。这些特征是使用 Commit Guru [71] 从这些项目中提取的。这些特定特征的使用在先前的研究 [18]、[70] 中已普遍存在并得到认可。Commit Guru 是一种基于 2015 年 ESEC/FSE 论文开发的公开可用工具，已在众多研究 [75]、[76] 中使用。这 14 个特征成为了本实证研究中使用的独立属性。

根据纳加潘（Nagappan）和鲍尔（Ball）的研究结果，我们通过按 LT（变更前文件中的代码行数）以及 LT 和进行区分，创建了相对变更率以及标准化的 LA（新增代码行数）和 LD（删除代码行数）特征。

表 2: Commit Guru 工具 [70]、[71] 从 GitHub 代码库中挖掘出的 14 个提交级别特征

Dimension	Feature	Definition
Diffusion	NS	Number of modified subsystems
	ND	Number of modified directories
	NF	Number of modified files
	ENTROPY	Distribution of modified code across each file
Size	LA	Lines of code added
	LD	Lines of code deleted
	LT	Lines of code in a file before the change
Purpose	FIX	Whether the change is defect Changes that fixing the defect are more likely to introduce more defects fixing ?
History	NDEV	#developers changing modified files
	AGE	Mean time from last to the current change
	NUC	#changes to modified files before
Experience	EXP	Developer experience
	REXP	Recent developer experience
	SEXP	Developer experience on a subsystem

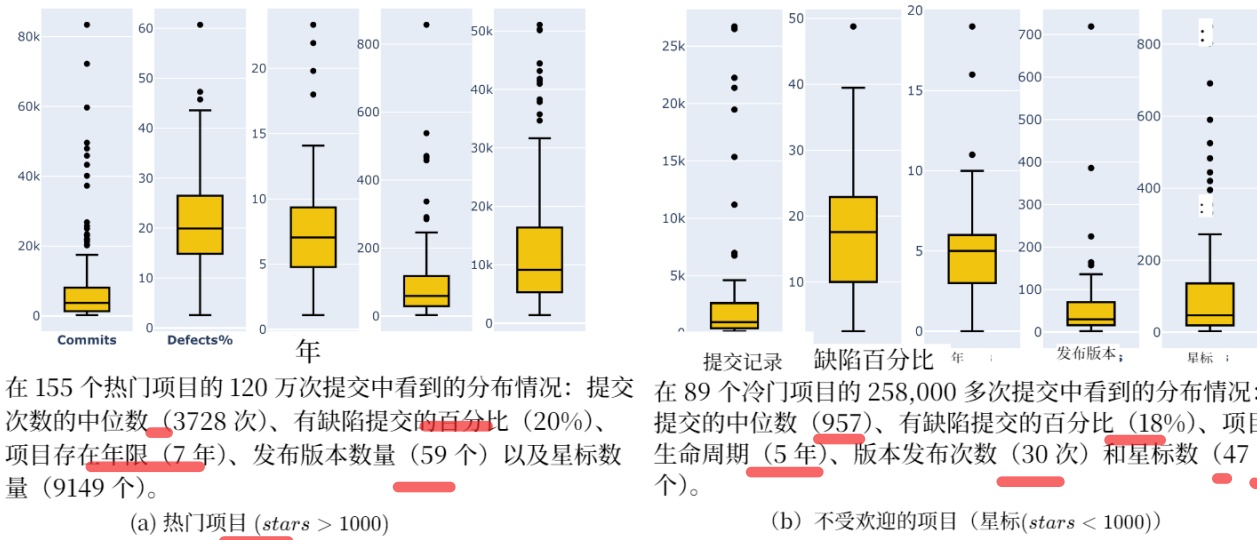


图 2：数据分布。

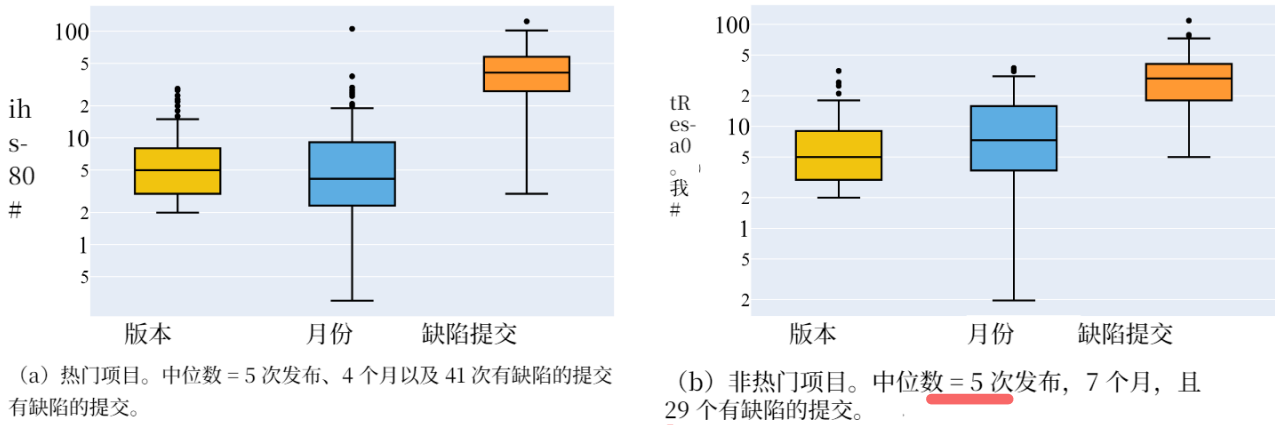


图 3：前 150 次提交中的分布情况。

Nature	Type	Method	Pre-processing	# Features (columns)	# of commits (rows)
Data-hungry	CPDP	Bellwether [37]	CFS, SMOTE and steps illustrated in §4.1	Selected by CFS.	All commits from the identified cross project.
		TCA+ [12]	SMOTE	5 components with linear kernel (data supplied with all features)	Pick first 150 and last 150 commits from the bellwether project.
Data-lite	CPDP	$*E_{Bellwether}^+$	Steps illustrated in §4.1 in [70], [74], [75]	LA, LT = lines added, lines of code in file before change	Sample equal number of defective and clean commits as available in the first 150 commits (not exceeding 25 each).
		$*E_{TCA+}^+$	Steps illustrated in §4.1	2 components with linear kernel (data supplied only with LA and LT)	
Data-lite	WPDP	E [54] $*E^+$	CFS and steps illustrated in §4.1 Steps illustrated in §4.1	Selected by CFS LA and LT	(same as above)

关键：所有轻量级数据采样方法均用阴影表示 ()，本研究提出的策略用 * 表示。

表 3：该表列出了三种基线方法（其中两种是 CPDP，另一种是 WPDP）以及三种轻量级数据变体，以支持本研究。请注意，这些方法中有许多采用了文献 [70]、[74]、[75] 中建议的步骤。这些步骤在 4.1 节中进行讨论。此外，关于 CFS 的讨论，请参见 4.2 节。

NUC（修改文件之前的唯一变更数量）除以 NF（修改文件的数量）[74]，[77]。同样，我们舍弃了 ND（修改目录的数量）和 REXP（开发者近期经验），因为龟井等人发现 NF 和 ND 与 REXP 以及 EXP（开发者经验）相关。最后，我们对其余特征（布尔变量“FIX”除外）进行对数变换以处理偏态问题 [78]。

与 SZZ 算法 [79] 的回溯方法类似，Commit Guru 在内部将每个提交标记为“有缺陷”（基于某些与缺陷相关的关键字）或“无缺陷”（否则）。我们在代码中回溯，以找到与该提交相关的更改。

这项实证研究使用了三组算法：

• 4.1.1 节中描述的六种分类算法；

- 4.2 节中描述的（针对某些采样策略的）预处理算法；
- 这七种抽样方法在 § 4.2.1 中描述。

4.1.1 分类器

我们使用了早期 WPDP 研究 [54] 中采用的全部六种分类器。这六种分类器在软件工程文献中很常见，它们选自 Ghotra 等人 [80] 的研究成果，该研究对 30 多种缺陷预测算法进行了四个等级的广泛比较。

这些分类器包括：

- 逻辑回归 (LR)；
- 最近邻算法 (KNN)（最少 5 个邻居）；决策树 (DT)；
- 随机森林 (RF)
- 国家贝叶斯；
- 支持向量机 (SVM)

4.1.2 预处理程序

为了构建模型，有多种数据采样方法，没有一篇论文能涵盖所有方法。基于我们对表 1 中论文的研读，[54] 中发表的不同方法的一些频次统计以及我们自己的工程判断，以下方法代表了该领域之前的许多研究：

- [74]、[77]、[78] 建议的步骤（见 § 4.1）；
- 基于相关性的特征选择（CFS，见 § 4.2）；
- 合成少数类过采样（SMOTE，见 4.2 节）。

并非所有预处理步骤的组合都是有效的。表 3 展示了本文中使用的每种采样策略所应用的预处理类型。

（技术说明：为了记录我们避免了对有效性的常见威胁，我们在此声明，我们仅对训练数据应用了 SMOTE 算法，而未对测试数据应用 [68]。）

4.2 算法

下一节将提供关于我们的预处理器和其他算法的更多细节。

基于相关性的特征选择（CFS）：CFS 是 Hall [81] 提出的一种常用的特征选择技术，常用于构建有监督的缺陷预测模型。

模型，尤其是在构建缺陷预测器时 [77]。一种基于启发式的技术，用于逐步评估特征子集。在内部，应用最佳优先搜索来识别有影响力但相互不相关的特征集。然而，它应该与目标（分类）相关。下面我们展示每个子集是如何计算的：

$$merits = krcf / \sqrt{k + k(k-1)rf} \text{ where}$$

- 优点是包含 k 个特征的某个子集 s 的价值；
- rcf 是一个分数，用于阐明该组特征与类别的关联；
- rf 是用于包含 s 中事物之间的平均得分和关联的组件，其中 rcf 应该很大，且 rf 。

合成少数类过采样技术（SMOTE）：由乔拉等人 [82] 提出的一种采样技术，用于处理类别不平衡问题。因为当训练集中的干净提交和有缺陷提交（或模块、记录等）数量不相等时，分类器可能难以识别目标类别。在缺陷预测领域，许多研究人员推荐使用 SMOTE [68]、[83]。

SMOTE 使用 K 近邻算法（至少需要五条提交记录）来推断和合成人工数据点（提交记录），以平衡训练集。如前所述，表 3 中列出的每种采样策略并非都需要 SMOTE。基于早期“E”的策略，根据定义，其缺陷提交记录和无缺陷提交记录的数量相等，因此是平衡的。

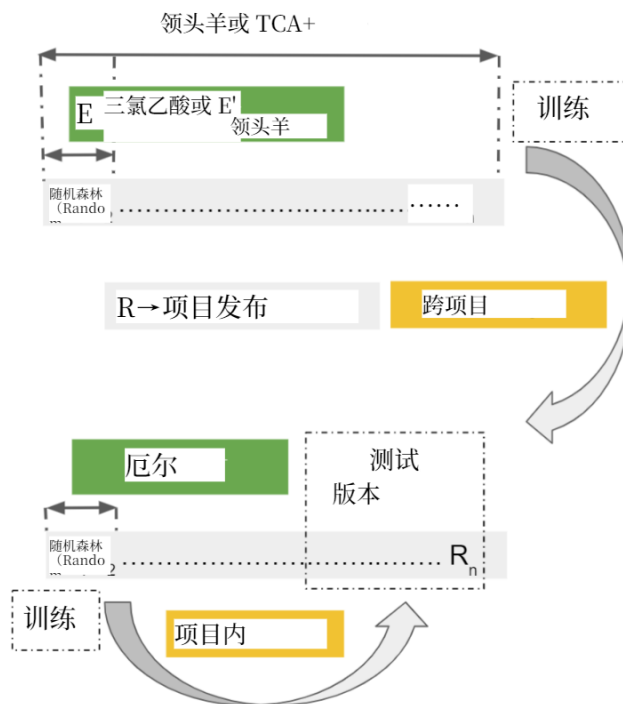


图 4：用于评估表 3 中列出的采样策略的方法的可视化地图。

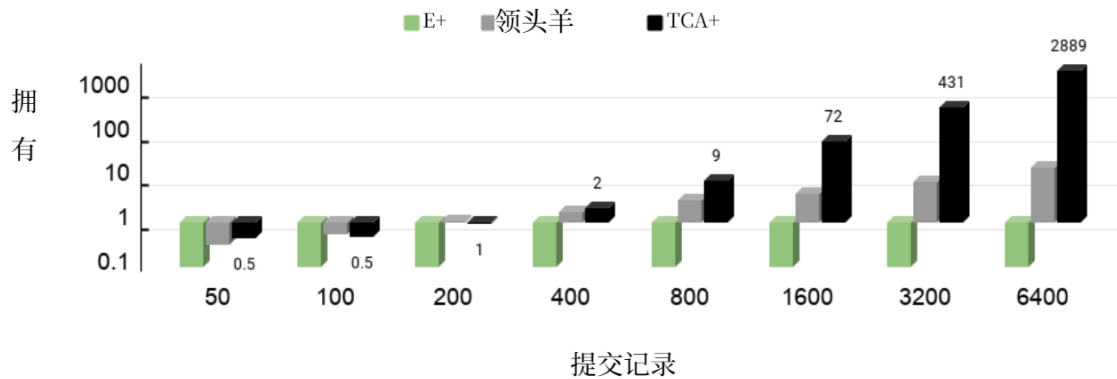


图 5: 基于提交次数, 不同采样策略训练模型所需的时间。

4.2.1 采样方法

图 4 展示了一组选项, 我们可以从中生成大量的采样选项。例如, 训练数据可以来自内部, 也可以从其他项目中交叉提取。此外, 在领先指标法中, 我们会探索并删减大部分训练数据 (仅使用能产生优于所有其他项目模型的模型的项目)。

为了找到有代表性的采样技术, 我们查阅了表 1 中列出的论文, 以了解标准采样策略、CPDP 技术、项目和特征。我们发现, 在过去几十年中有大量关于 CPDP 技术的报道; 例如, Sousuke 最近 (2020 年) 的研究探索了 24 种 CPDP 方法 [3]。在 WPDP 的背景下, Shrikanth 等人确定了许多构建预测模型的采样策略 (基于最新发布的数据、最近几个月的数据以及所有过去的的数据) [54]。然而, 在 CPDP 的背景下, 我们只发现了两种 (“全部” 或 “部分”)。尽管如此, “全部” 或 “部分” 都需要大量数据。本研究的重点是检验在流行的 CPDP 方法中使用轻量级数据方法的有效性, 而不是对众多 CPDP 技术进行排名。我们选择了两种有代表性的 CPDP 技术, 特别是 TCA+ 和 Bellwether, 原因如下:

- TCA+ 是该领域基于 Nam 等人 [12] 开创性工作的一种活跃的 CPDP 技术。
- 贝尔韦瑟 (Bellwether), 这是克里希纳 (Krishna) 等人在近期 (2018 年) 提出的一种基线方法, 其表现优于 TCA+ [45]

我们在多种分类算法的 12000 多个版本上测试了这两种及以下技术。下面让我们来研究一下这些数据密集型技术。

标杆项目 (Bellwether): 标杆项目的所有提交记录都被用作训练数据 [45]。通过比较群体中每个项目与其他所有项目的预测性能得分来确定标杆项目。标杆迁移学习方法假定 “当一个社区致力于软件开发时, 就会存在一个模范项目, 称为标杆项目, 它可以为其他项目定义预测指标。[45]”。根据克里希纳 (Krishna) 等人的研究 [45], 找到标杆项目需要进行一项 $O(N^2)$ 研究, 这种方法很难扩展到数百或数千个项目。

迁移成分分析 (TCA+): Pan 等人提出了一种域适应技术, 该技术能够实现源域和目标域之间的信息转换 [84]。在此基础上, Nam 等人在 TCA 之上叠加了许多归一化规则, 从而形成了 TCA+[85]。根据源项目和目标项目之间数据集特征的相似性, 应用特定的归一化规则。Rahul 和 Menzies [45] 报告称, TCA+ 的表现优于其他迁移方法, 即迁移朴素贝叶斯 [6] 和价值认知增强学习器 [32]。

使用 TCA+ 进行迁移学习的方法如下:

- 从所有的标杆项目中随机选择一个标杆项目。
- 图 5 展示了对于 TCA+ 而言, 随着训练提交次数的增加, TCA+ 的运行时成本急剧上升。对全部六个分类器在 12000 多个版本中重复进行测试在实际中并不可行。对于基于 Bellwether 的策略, 由于其使用固定的所有提交集, 我们至少可以缓存预测器。而对于 TCA+, 缓存是不可能的, 因为该方法是根据训练和测试提交来决定转换规则的。
- 为了在不掩盖 TCA+ 对数据的高需求的情况下管理运行时的复杂性, 我们采取以下措施:
 - 我们只限定使用 300 个训练提交记录 (150 个提交记录的两倍), 而不是考虑项目的所有提交记录。
 - 但选取前 300 次提交会忽略项目的最新数据, 可能无法代表项目的生命周期。因此, 在这 300 次提交中, 我们从项目数据里抽样最早的 150 次提交和最晚 (最近) 的 150 次提交。不过, 需要注意的是, 在 WPDP 的背景下, 施里坎特等人表明, 使用项目的所有提交来预测缺陷是没有必要的 [54]。

对于表 3 中列出的与 TCA+ 相关的策略, 我们复用了近藤等人在其复现代码包 3 中的实现。

3. <https://sailhome.cs.queensu.ca/replication/featred-vs-featseldefectpred/>

等人, 这是一篇发表于 EMSE' 19 会议, 关于缺陷预测模型中特征约简技术的文章 [77]。

接下来, 我们将讨论上述两种技术的两种轻量级数据变体。

早期风向标 ($E_{Bellwether}^+$) 并非使用风向标项目中的所有提交, 而是精心挑选了一小部分训练数据。具体来说, 从最早的 150 次提交中随机抽取 25 个有缺陷的变更和 25 个无缺陷的变更 (这是 Shrikanth 等人提出的一种方法 [54])。除了两个基于规模的特征 “LA” 和 “LT” 之外, 所有的数据特征都被去除。选择这两个特征的理由将在 4.5 节中讨论。

早期 TCA+ (E_{TCA+}) 此提议的技术与 TCA+ 类似, 除了 “采样方法”。该方法并非使用所选项目中的所有提交, 而是精心挑选一小部分训练数据。具体来说, 从最早的 150 次提交中随机抽取 25 个有缺陷的变更和 25 个无缺陷的变更 (这是 Shrikanth 等人在文献 [54] 中提出的方法)。除了 “LA” 和 “LT” 之外, 所有的数据特征都被去除。选择这两个特征的理由将在 4.5 节中讨论。

与 TCA+ 类似, 但 “采样方法” 不同。它并非使用所选项目中的所有提交, 而是精心挑选了一小部分训练数据。具体来说, 从最早的 150 次提交中随机抽取 25 个有缺陷的变更和 25 个无缺陷的变更 (这是 Shrikanth 等人提出的一种方法 [54])。除了两个基于规模的特征 “LA” 和 “LT” 外, 所有数据特征均被去除。选择这两个特征的理由将在 4.5 节中讨论。

然后, 为了将 CPDP 与 WPDP 技术进行比较, 我们选择了 E, 该方法由 Shrikanth 等人在文献 [54] 中提出, 并被证明优于使用所有过去提交或近期提交的预测器。为了检验 E 是否像我们对 $E_{Bellwether}^+$ 所做的那样, 仅用两个特征就足够了, 我们创建了 E^+ 。

(E) 训练数据是通过从所研究项目内做出的前 150 个变更中随机抽取 25 个有缺陷的变更和 25 个无缺陷的变更来精心整理的 [54]。

早期少量特征采样 (E^+) 训练数据的整理方式与 E 相同, 但仅限于表 2 中列出的两个基于尺寸的特征 “LA” 和 “LT”。其基本原理将在 § 5 中讨论。)。

未由 “+” 特别标注的采样方法意味着, 在构建预测器时, 会根据需要使用 4.2 节中讨论的基于相关性的特征选择 (CFS) 方法来选择特征。

最后, 我们在此指出, 这项工作并非要对诸如 TCA+ 或 Bellwether 等不同的 CPDP 技术进行排名, 而是要 “检验” 这些流行的技术在轻量级数据方法下是否足够有效 (或表现更佳)。

4.3 评估标准

我们使用基线研究中采用的全部七项标准, 来评估利用各种采样策略构建的预测模型。该研究参考了缺陷预测文献中广泛使用的度量指标 [1]、[21]、[70]、[75]、[83]、[86]–[93]。

关于以下七种预测性能指标的说明:

- 除了初始误报数量外, 其他六项指标的取值范围均为 0 到 1;
- D2H、IFA、Brier、PF 这些标准需要最小化, 也就是说, 对于这些标准, 数值越小越好。
- 对于其中的三个指标, 即 AUC、召回率和 G 值, 需要将其最大化, 也就是说, 对于这些指标, 数值越高越好。

先前的研究表明, 对于不平衡数据, 精度存在显著问题。我们在评估中未纳入这一点 [86]。

在列出这些标准之前, 我们注意到, 在实践中, 至少对于此处所探究的数据而言, 其中许多标准并无参考价值。例如, IFA 在我们所有的处理方法中得出的结果在统计上并无显著差异。同样, 我们所有更好的方法都会实现相似的 G 值 (因此该指标对于确定哪种处理方法 “最佳” 并非至关重要)。

4.3.1 布赖尔 (Brier)

布里尔分数 (Brier) 是一种绝对预测准确率度量。众多缺陷预测论文 [1]、[75]、[83]、[89] 都认可这种度量。设 c 为测试提交的总数。设 y_i 为 1 (对于有缺陷的提交), 否则为 0。设 $\hat{y}_i$ 为提交存在缺陷的概率 (根据 scikit-learn 库 [94] 中的损失函数计算得出)。

然后:

$$Brier = \frac{1}{c} \sum_{t=1}^C (y_i - \hat{y}_i)^2 \quad (1)$$

4.3.2 初始误报数量 (IFA)

根据帕宁 (Parnin) 和奥尔索 (Orso) [95] 的观察, 如果开发人员遇到许多最初的误报, 他们就会对这种分析失去信任。因此, 只需按照易出现缺陷的概率顺序对提交进行排序后, 统计遇到的误报数量。首次发现实际警报前的误报数量即为首次误报数 (IFA)。

4.3.3 召回率

召回率是已检查出的缺陷提交数除以所有缺陷提交数。

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives}$$

4.3.4 误报率 (PF)

误报率 (PF) 是指被预测为有缺陷的干净提交数量与所有有缺陷的提交数量 (无论分类情况如何) 之间的比率。

$$PF = \frac{FalsePositives}{FalsePositives + TrueNegatives}$$

4.3.5 受试者工作特征曲线下面积 (AUC)

它仅仅是误报率和真正率之间曲线下的面积。

4.3.6 与天的距离 (D2H)

D2H 或 “距天堂的距离” 是通过将召回率和误报率 (PF) 这两个指标进行汇总计算得出的。其中, “天堂” 是指一个具有 $Recall = 1 \& PF = 0$ [96] 的地方。

$$D2H = \frac{\sqrt{(1-Recall)^2 + (0-PF)^2}}{\sqrt{2}}$$

4.3.7 G 度量 (GM)

几何平均值 (GM) 计算为误报率 (PF) 的互补值与召回率之间的调和平均值。其计算方式如下：

$$G - Measure = \frac{2 * Recall * (1 - PF)}{Recall + (1 - PF)}$$

通用指标 (GM) 和 D2H 本质上结合了相同的两个度量，即召回率 (Recall) 和 PF。尽管如此，我们仍然采用它们，因为它们文献中已分别被认可使用。值得注意的是，从文献 [54] 以及本文后面第 5 节所示的结果来看，在通用指标 (GM) 上取得好的结果并不一定意味着在 D2H 上也能取得好的结果 (反之亦然)。

由于分类过程的性质，某些标准总会得出相互矛盾的结果：

- 一个分类器可能只需将所有测试提交标记为有缺陷，就能轻松达到 100% 的召回率。但这样做会带来一个副作用，即该方法会导致较高的 PF：
- 其次，分类器可能会将所有测试提交都归类为无问题，以显示 0% 的 PF，但这种方法的召回率会非常低。
- 最后，布里尔分数 (Brier) 和召回率 (Recall) 也是对立的，因为降低损失函数意味着遗漏一些结论，从而降低召回率。

4.4 统计检验

使用表 3 中列出的每种采样策略构建的预测器，会在所有合适的 (未来) 项目版本上进行测试。然后，我们计算 § 4.3 中讨论的各项评估标准。接着，每个分数会按采样策略和分类器的组合进行分组并导出。

需要重申的是，每个总体都是特定评估分数的集合。此外，不同总体可能具有相同的中位数，但分布却完全不同。因此，为了对这些评估分数总体进行排序，我们采用了米塔斯等人在《IEEE 软件工程汇刊》(TSE) 2013 年论文 [97] 中推荐的斯科特 - 诺特检验 (Scott-Knott test)。此过程是一种自上而下的双向聚类方法，用于在特定评估指标下，对使用不同采样策略和分类器创建的不同预测变量进行排序。该方法根据中位数对由 l 个评估分数组成的列表 ls 进行排序。接下来，它将 l 拆分为子列表 m 和 n 。这样做是为了使划分前后观测性能差异的期望值最大化。

对于大小分别为 ls 、 ms 、 ns 的列表 l 、 m 、 n ，其中 $l = m \cup n$ ，“最佳”划分可使 $E(\Delta)$ 最大化；即划分前后预期平均值的差异：

$$E(\Delta) = \frac{ms}{ls} abs(m.\mu - l.\mu)^2 + \frac{ns}{ls} abs(n.\mu - l.\mu)^2$$

此外，为避免出现具有统计学显著结果的“小效应”，采用了自举法与 Vargha 和 Delaney 提出的 A12 效应量检验法 [98] 相结合的方法。

重要说明：我们对所有七个评估标准分别应用 Scott-Knott 检验；也就是说，在计算排名时，我们针对 (例如) 召回率，对 PF 单独进行计算。

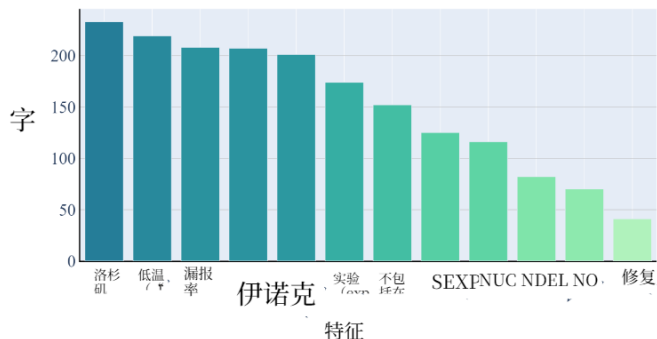


图 6: CFS 特征选择器选择的特征频率 (仅使用前 150 次提交)，在 240 个项目中使用 E 进行采样

4.5 其他细节

我们希望不仅报告性能分数，还提供一些关于生成这些分数的模型的细节。为此，我们将利用领头羊效应。具体来说，我们将

- 找到最重要的特征，然后
- 将它们用于代表我们大部分数据的领头羊样本中。

4.5.1 寻找重要特征：

什里坎特等人的早期抽样规则“E”已被证明适用于 WPDp [54]。鉴于我们正在从这样一个样本构建模型，不禁要问：“那个小样本能产生一个简洁的模型吗？”请注意，如果确实如此，那么我们就可以清晰地报告哪些因素对缺陷影响最大。

图 6 展示了 CFS 算法 (见 § 4.2) 在我们所有 240 个项目 (仅使用前 150 次提交) 选择的特征频率。为了从该空间中选择“最佳”特征，我们使用排名为 $top 1 < x \leq 12$ 的特征构建模型，当使用排名为 $x+1$ 的特征表现不比排名为 x 的特征更好时停止 (在此，我们测试的是前 150 次提交之后发生的所有版本)。该过程表明，从两个大小排名特征中学习到的模型与其他任何模型的表现一样好：

- LA: 新增代码行数
- LT: 变更前文件中的代码行数

4.5.2 寻找具有代表性的指标：

对于我们的每个项目，仅使用洛杉矶 (LA) 和伦敦 (LT) 的数据，我们找到了一个模型，并在其他项目上衡量了其性能。如果该模型的中位数性能达到“召回率”70% 以上且“误报率 (PF)”低于 30%，则该模型被认为是“令人满意的”。图 7 的结果支持了使用早期生命周期数据的做法，图 7 显示，通过 E^+ 方法找到的模型中，“令人满意”的模型数量比使用所有数据找到的模型更多。

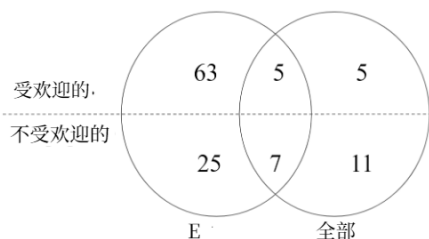


图 7: 240 个项目中, 两种不同抽样策略独立识别出的“满意”风向标数量。

该数据。这种影响在热门项目中尤为明显 4。

在本文后面部分, 我们将展示以这种方式构建的模型与其他各种处理方法的比较结果。

5 结果

表 4 展示了我们的结果。在该表的第一行中, “+” 和 “-” 分别表示需要最大化或最小化的标准。所有其他行展示了采样策略和分类器的组合。绿色单元格表示“轻数据”方法(而所有其他行表示“数据密集型”方法)。“获胜次数”列(见第 1 列)统计特定采样策略/分类器获得最高分的频率。灰色标记的单元格与最佳结果具有相同的最高排名(这些排名由 § 4.4 中描述的 Scott-Knott 算法确定)。

观察灰色单元格的模式, 部分标准 (IFA、Brier 和 PF) 结果证明并无参考价值:

- 结果表明, IFA 几乎总是打成平手(该列中大量的灰色单元格就是证明)。
- 此外, 获得最佳“PF”和最佳“Brier”是以较低的“召回率”结果为代价的(参见 Bellwether+SVM 或 RF, 在表格往下三分之一处)。

看一下表格的第一行(黄色标记处), 我们可以看到一个跨公司早期生命周期模型。

(Bellwether, 逻辑回归) 在所有标准上都取得了最佳结果(布里尔分数除外)。因此我们说:

结论 1: 对于缺陷预测, 如果你能够获取许多其他项目的数据, 那么使用仅通过前 150 次提交中的 LA 和 LT 识别并构建的任何一个指标。

当然, 并非所有项目都能获取其他许多项目的数据。在这种情况下, 我们注意到表 4 第 5 行的结果显示 ((E^+, LR))。这种项目内处理在统计上不如我们排名最高的处理方法。

- 显然, 这种分析可能过度依赖于选择“令人满意的”领头羊项目的“神奇数字”, 即 70% 的“召回率”和低于 30% 的“误报率”。但是我们使用了 12 个不同的领头羊项目以几乎相同的频率运行实验(见图 7 中间的 7 个冷门 + 5 个热门领头羊项目)。使用 4.4 节对这 12 个项目进行的统计分析表明: (a) 它们彼此不分伯仲; (b) 其表现与本文探索的其他迁移学习方法一样好, 甚至更好。

报告称, 它与其他三种接近顶级的治疗方法并列, 其表现与我们最好的治疗方法相近。因此我们说:

结论 2: 对于缺陷预测, 如果你只能访问自己的项目, 等待 150 次提交, 然后使用仅包含 LA 和 LT 特征的自有数据构建模型。

为完整起见, 我们再提供两条观察结果:

- 首先, 之前最先进的迁移学习方法 (Bellwether, TCA+) 似乎低于 $E_{Bellwether}^+$ 。这一结果进一步证实了早期生命周期推理的有效性。
- 其次, 本文是对先前工作 [54] 的扩展, 该工作使用了由 CFS 选择的特征。如该表第 20 行的黄线所示, 通过 LA 和 LT 学习的模型在每个版本上的表现都优于使用 CFS 选择的特征所学习的模型。因此, 我们不再推荐我们先前结果中的这一部分。

5.1 效度威胁

5.1.1 采样偏差

结论的普遍适用性将取决于所考虑的示例; 也就是说, 此处关键的内容可能并非放之四海而皆准。尽管此类实证研究的普遍做法是使用热门的操作系统相关 GitHub 项目, 但我们通过纳入冷门项目拓宽了研究范围。然而, 所有这些项目都是用多种编程语言为不同领域开发的重要工程项目。值得注意的是, 在这两类项目中, 所得到的经验教训并无差异。

5.1.2 学习者偏差

为了通过全对实验(每个项目都在 12000 多个版本上进行测试)识别领头羊项目, 我们仅使用了一种分类器, 即“逻辑回归”。也许其他分类器可能会将其他项目认定为领头羊项目。仅使用一种分类器可能不会构成威胁, 原因有以下两点。首先, 在基线研究 [54] 中推荐使用逻辑回归。其次, 在表 4 中, 我们将我们的结果与多种分类器进行了比较, 基于逻辑回归的预测器与 § 4.1.1 中列出的其他五种分类器相当, 甚至更优。

此外, 实证研究只能聚焦于少数具有代表性的分类器。因此, 我们选择了六种分类器(逻辑回归、最近邻、决策树、随机森林和朴素贝叶斯)。这六种分类器涵盖了广泛的分类算法 [80]。

5.1.3 评估偏差

本文同时使用“召回率”(Recall)和“预测力度”(PF)来识别领先指标, 并使用七种评估指标(召回率、预测力度、IFA、Brier、GM、D2H 和 AUC)对这些策略进行广泛比较。缺陷预测中其他广泛使用的指标是准确率和 F 值。然而, 如前文所述, 那些依赖阈值的指标在处理不平衡数据时存在问题 [86]。

表 4: 在 155 个热门和 85 个冷门 GitHub 项目的 12379 个适用版本中测试的 36 个缺陷预测模型 (包括 WPDP 和 CPDP)。第一行中, “+” 和 “-” 分别表示需要最大化或最小化的标准。其他所有行展示了采样策略和分类器的组合。绿色单元格表示 “轻数据” 方法 (其他所有行表示 “数据密集型” 方法)。灰色标记的单元格与最佳结果具有相同的最高排名 (这些排名由 § 4.4 中描述的 Scott-Knott 算法确定)。“获胜次数” 列 (见第 1 列) 统计特定采样策略 / 分类器获得最高分的频率。

Type	Policy	Classifier	Wins	Recall+	PF-	AUC+	D2H-	Brier-	GM+	IFA-	
CPDP	$E^+_{Bellwether}$	LR	5	0.85	0.34	0.73	0.32	0.31	0.76	1.0	← best transfer learning
CPDP	$E^+_{Bellwether}$	SVM		0.9	0.4	0.71	0.35	0.35	0.74	1.0	
CPDP	E^+_{TCA+}	SVM		0.88	0.39	0.7	0.35	0.34	0.74	1.0	
CPDP	E^+_{TCA+}	LR	4	0.82	0.33	0.72	0.32	0.31	0.75	1.0	
WPDP	E^+	LR		0.77	0.26	0.73	0.31	0.27	0.73	1.0	← best within-project learning
WPDP	E^+	NB		0.89	0.44	0.68	0.39	0.38	0.71	2.0	
CPDP	E^+_{TCA+}	KNN		0.8	0.33	0.7	0.35	0.32	0.72	1.0	
WPDP	E^+	SVM		0.8	0.29	0.71	0.35	0.29	0.71	1.0	
CPDP	$E^+_{Bellwether}$	KNN	3	0.78	0.3	0.7	0.34	0.3	0.72	1.0	
CPDP	E^+_{TCA+}	RF		0.75	0.32	0.69	0.35	0.32	0.71	1.0	
CPDP	$Bellwether$	SVM		0.22	0.04	0.58	0.56	0.19	0.26	1.0	
CPDP	$Bellwether$	RF		0.2	0.03	0.57	0.57	0.19	0.23	1.0	
CPDP	$E^+_{Bellwether}$	NB		0.88	0.5	0.64	0.42	0.42	0.67	2.0	
CPDP	E^+_{TCA+}	NB	2	0.83	0.44	0.67	0.39	0.39	0.7	2.0	
WPDP	E^+	KNN		0.7	0.26	0.7	0.34	0.29	0.68	1.0	
CPDP	$Bellwether$	KNN		0.25	0.06	0.59	0.53	0.2	0.29	1.0	
CPDP	$Bellwether$	LR		0.77	0.33	0.67	0.39	0.32	0.66	1.0	
CPDP	E^+_{TCA+}	DT		0.71	0.32	0.67	0.36	0.33	0.68	2.0	
WPDP	E	SVM		0.71	0.31	0.68	0.36	0.31	0.67	1.0	
CPDP	$E^+_{Bellwether}$	RF		0.71	0.29	0.68	0.36	0.3	0.68	1.0	
CPDP	$TCA+$	LR		0.7	0.5	0.62	0.42	0.45	0.64	2.0	
WPDP	E	LR		0.7	0.31	0.67	0.37	0.32	0.66	1.0	← prior result [54]
WPDP	E	KNN		0.67	0.31	0.66	0.39	0.33	0.63	1.0	
CPDP	$E^+_{Bellwether}$	DT		0.67	0.3	0.65	0.38	0.33	0.64	1.0	
WPDP	E^+	RF		0.67	0.27	0.68	0.36	0.29	0.65	1.0	
WPDP	E^+	DT	1	0.64	0.3	0.65	0.38	0.32	0.62	1.0	
WPDP	E	RF		0.62	0.27	0.63	0.42	0.32	0.56	2.0	
WPDP	E	DT		0.6	0.36	0.58	0.48	0.38	0.52	2.0	
CPDP	$TCA+$	DT		0.5	0.33	0.56	0.48	0.38	0.48	2.0	
CPDP	$TCA+$	RF		0.5	0.3	0.58	0.48	0.36	0.49	2.0	
CPDP	$TCA+$	SVM		0.5	0.22	0.61	0.45	0.31	0.51	2.0	
CPDP	$Bellwether$	NB		0.5	0.15	0.64	0.41	0.25	0.53	1.0	
CPDP	$TCA+$	KNN		0.38	0.18	0.58	0.49	0.29	0.41	2.0	
CPDP	$Bellwether$	DT		0.33	0.13	0.59	0.48	0.25	0.38	1.0	
WPDP	E	NB	0	0.55	0.31	0.54	0.57	0.37	0.37	3.0	
CPDP	$TCA+$	NB		0.22	0.19	0.5	0.61	0.33	0.23	3.0	

关键信息: 所有轻量级数据采样方法均以阴影显示 ()。

5.1.4 输入偏差

我们所有提议的抽样策略都是从项目的前 150 次提交中随机抽取 50 次提交。照此而言, 不同的执行可能会产生不同的结果, 这一点或许属实。然而, 这并非问题所在, 因为我们的结论是基于超过 12000 个版本的大样本量得出的。

6 讨论

6.1 管理启示

此前许多研究都难以得出适用于多个项目的稳定结论。门齐斯等人警告称, 许多全局经验 (概括) 在局部并不成立 [4]。也就是说, 我们看到的 数据越多, 从这些数据中学习得到的模型中可能出现的例外和特殊情况就越多。软件工程领域中这种结论的不稳定性在 [4]、[66]、[99] 中多有记载。

这就是我们能做到的最好程度了吗? 理想情况下, 软件工程研究可以提供稳定的通用缺陷预测原则 (如上述那些原则), 以指导项目管理、软件标准制定、教育、工具开发以及相关立法。

软件。这种结论稳定性将在信任、洞察、训练和工具开发方面带来益处。

信任: 结论的不稳定性会让项目经理感到不安。哈桑 [100] 警告称, 如果软件分析的结果不断变化, 管理人员就会对其失去信任。但如果我们能找到稳定的结论 (例如, 使用早期生命周期指标), 这将为项目经理在许多问题上提供明确的指导方针, 包括: (a) 何时应该检查某个模块; (b) 何时应该重构模块; 以及 (c) 决定将重点放在哪些昂贵的测试程序上。

洞察: 索耶等人断言, 洞察对于推动商业举措至关重要 [101]。

从金等人 [102] 的角度来看, 软件分析是一种获取富有成效的洞察的方法, 这些洞察可指导从业者实现软件开发目标, 而对于谭等人 [103] 来说, 此类洞察是核心目标。从从业者的角度来看, 伯德等人 [104] 报告称, 当用户对软件分析模型做出响应时, 就会产生洞察。频繁的模式更新可能会耗尽用户从新数据中得出可靠结论的能力。在这方面, 我们注意到, 早期发现的生命周期模型将在项目生命周期的大部分时间内, 成为稳定的洞察来源。

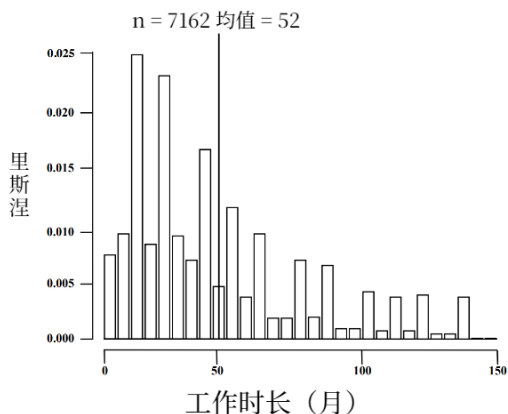


图 8：特定项目的工作时长直方图；数据来自 [105]。数据来源：Facebook、eBay、苹果、3M、英特尔和摩托罗拉。

工具开发与培训：此前 [99] 我们曾警示，不稳定的模型会增加新手软件工程师入职的难度。由于不清楚哪些因素对本地项目影由于我们只需要前 150 次提交来确定一个合适的项目，许多处于早期阶段的项目可能会被纳入可迁移项目库中。重要的是，图 7 显示，当使用 E^+ 进行采样时，“多 30%”的项目能在“更短的时间”内符合要求。领头羊的选择并不重要；也许在实际操作中，从业者无需继续寻找领头羊，一旦找到第一个领头羊就可以提前终止。这也意味着隐私问题减少，从业者可以更自由地分享更少的数据。我们方法的数据精简特性还能帮助更快地评估像 TCA + 这样复杂的技术。

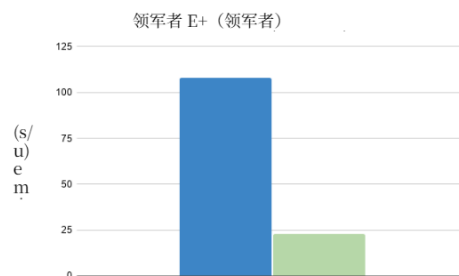


图 9：使用 Bellwether 和 E^+ Bellwether 策略在 240 个项目识别合格的风向标项目所需的时间。请注意，在我们的实验中，我们在基于云计算的多核 CPU 集群上运行。为了收集此处的数据（假设我们在单核机器上运行），我们对实验设备中所有核心的 CPU 时间进行了求和。我们断言这个总和是有效的，因为在我们的实验中，核心之间几乎没有通信（除了在最后阶段汇总结果）。

6.3 研究意义

我们已经表明，当缺陷数据包含信息时，这些信息可能在项目历史记录的一小部分中最为密集。虽然我们尚未证明其他类型的软件工程数据也有相同的密度效应，但我们认为，“我们是否一直在从数据的错误部分学习？”现在至少是一个悬而未决的问题。

至于其他未来的工作，鉴于轻量级数据方法的简便性，我们渴望探索超参数优化 (HPO) [106] - [108]。由于计算复杂性，超参数优化在软件分析中通常未被采用。也许通过在生命周期早期仅关注小样本数据，可以避免这种复杂性。

最后，我们现在应该将在一个项目中对所有数据进行推理视为一种潜在的方法错误。在缺陷预测的特定情况下，我们现在必须重新审视仅基于生命周期后期数据得出的任何结论。这样的结论有很多例子。例如：

黄 (Hoang) 等人表示：“我们假设较旧的提交更改可能具有对最新提交不再产生影响的特征” [109]。

此外，在缺陷预测中，常见的做法是进行“近期验证”，即预测器在先期一两个版本上训练后，在最新版本上进行测试 [1]、[75]、[107]、[110]。

更普遍地说，在研究人员只专注于生命周期后期的数据之前，他们首先需要检查（例如）错误提交数据在整个生命周期中出现的频率是否相同。

致谢

这项工作部分得到了美国国家科学基金会计算机与信息科学与工程学部 (NSF-CISE) 编号为 1908762 的资助。

参考文献

- [1] S. 麦金托什和 Y. 龟井, “引发修复的变更是否是移动靶标? 一项关于即时缺陷预测的纵向案例研究”, 《IEEE 软件工程汇刊》, 第 44 卷, 第 5 期, 第 412 - 428 页, 2017 年。
- [2] F. 拉赫曼、D. 波斯内特、I. 赫赖斯和 P. 德万布, 《缺陷预测中的样本量与偏差》, 载于《2013 年第 9 届软件工程基础联合会议论文集》。美国计算机协会, 2013 年, 第 147 - 157 页。
- [3] S. 天崎, 《跨版本缺陷预测: 使用历史数据、跨项目数据, 还是两者皆用?》, 《实证软件工程》, 第 1 - 23 页, 2020 年。
- [4] T. 孟席斯、A. 布彻、A. 马库斯、T. 齐默尔曼和 D. 科克, 《工作量估计与缺陷预测的局部模型与全局模型对比》, 发表于 2011 年第 26 届 IEEE/ACM 国际自动化软件工程会议 (ASE 2011)。IEEE, 2011 年, 第 343 - 351 页。
- [5] 李 M.、张 H.、吴 R. 和周志华, 《基于样本的主动与半监督学习软件缺陷预测》, 《自动化软件工程》, 2012 年, 第 19 卷, 第 2 期, 第 201 - 230 页。
- [6] Y. Ma, G. Luo, X. Zeng 和 A. Chen, “用于跨公司软件缺陷预测的迁移学习”, 《信息与软件技术》, 第 54 卷, 第 3 期, 第 248 - 256 页, 2012 年。
- [7] Z. He, F. Shu, Y. Yang, M. Li 和 Q. Wang, “跨项目缺陷预测可行性研究”, 《自动化软件工程》, 第 19 卷, 第 2 期, 第 167 - 199 页, 2012 年。
- [8] T. 孟席斯、A. 布彻、D. 科克、A. 马库斯、L. 莱曼、F. 舒尔、B. 图尔汗以及 T. 齐默尔曼, 《缺陷预测与工作量估计的局部经验与全局经验对比》, 《IEEE 软件工程汇刊》, 第 39 卷, 第 6 期, 第 822 - 834 页, 2012 年。
- [9] N. 贝滕堡、M. 纳加潘和 A. E. 哈桑, 《立足本地, 着眼全局: 改进缺陷与工作量预测模型》, 发表于 2012 年第 9 届 IEEE 软件仓库挖掘工作会议 (MSR)。IEEE, 2012 年, 第 60 - 69 页。
- [10] F. 拉赫曼、D. 波斯内特和 P. 德万布, “忆跨项目缺陷预测的‘不精确性’”, 载于《美国计算机协会软件工程基础第 20 届国际研讨会论文集》, 2012 年, 第 1 - 11 页。
- [11] B. 图尔汗、A. T. 米西尔利和 A. 贝内尔, 《混合项目数据对缺陷预测学习效果的实证评估》, 《信息与软件技术》, 2013 年, 第 55 卷, 第 6 期, 第 1101 - 1118 页。
- [12] J. 南、S. J. 潘和 S. 金, 《迁移缺陷学习》, 发表于 2013 年第 35 届国际软件工程大会 (ICSE)。IEEE, 2013 年, 第 382 - 391 页。
- [13] F. 彼得斯、T. 孟席斯和 A. 马库斯, 《更优的跨公司缺陷预测》, 发表于 2013 年第 10 届挖掘软件仓库工作会议 (MSR)。IEEE, 2013 年, 第 409 - 418 页。
- [14] G. 坎福拉、A. D. 卢西亚、M. 迪彭塔、R. 奥利韦托、A. 帕尼切拉和 S. 帕尼切拉, 《多目标跨项目缺陷预测》, 载于《2013 年电气与电子工程师协会第六届软件测试、验证与确认国际会议论文集》。电气与电子工程师协会, 2013 年, 第 252 - 261 页。
- [15] F. 彼得斯、T. 孟席斯、L. 龚和 H. 张, 《跨公司缺陷预测中的隐私与效用平衡》, 《IEEE 软件工程汇刊》, 第 39 卷, 第 8 期, 第 1054 - 1068 页, 2013 年。
- [16] S. 赫博尔德, 《跨项目缺陷预测的训练数据选择》, 载于《软件工程预测模型第九届国际会议论文集》, 2013 年, 第 1 - 10 页。
- [17] Z. 何、F. 彼得斯、T. 孟席斯和 Y. 杨, 《从开源项目中学习: 缺陷预测的实证研究》, 发表于 2013 年 ACM/IEEE 实证软件工程与测量国际研讨会。IEEE, 2013 年, 第 45 - 54 页。
- [18] F. 拉赫曼和 P. 德万布, 《过程指标为何以及如何更优》, 发表于 2013 年第 35 届国际软件工程会议 (ICSE)。IEEE, 2013 年, 第 432 - 441 页。
- [19] T. 福岛、Y. 龟井、S. 麦金托什、K. 山下和 N. 宇林, 《使用跨项目模型进行即时缺陷预测的实证研究》, 载于《第 11 届挖掘软件仓库工作会议论文集》, 2014 年, 第 172 - 181 页。
- [20] A. 帕尼切拉、R. 奥利韦托和 A. 德卢西亚, 《跨项目缺陷预测模型: 团结就是力量》, 发表于 2014 年软件演化周——IEEE 软件维护、再工程与逆向工程会议 (CSMR-WCRE)。IEEE, 2014 年, 第 164 - 173 页。
- [21] F. 张、A. 莫克斯、I. 凯万卢和 Y. 邹, “迈向构建通用缺陷预测模型”, 载于《第 11 届挖掘软件仓库工作会议论文集》, 2014 年, 第 182 - 191 页。
- [22] D. 柳、J.-I. 张和 J. baik, 《一种使用最近邻的混合实例选择方法用于跨项目缺陷预测》, 《计算机科学与技术学报》, 第 30 卷, 第 5 期, 第 969 - 980 页, 2015 年。
- [23] L. 陈、B. 方、Z. 尚和 Y. 唐, 《跨公司软件缺陷预测中的负样本约简》, 《信息与软件技术》, 第 62 卷, 第 67 - 77 页, 2015 年。
- [24] 张 Y.、罗 D.、夏 X. 和孙 J., “跨项目缺陷预测的分类器组合实证研究”, 发表于《2015 年 IEEE 第 39 届年度计算机软件与应用会议》第 2 卷。IEEE, 2015 年, 第 264 - 269 页。
- [25] F. 彼得斯、T. 孟席斯和 L. 莱曼, 《Lace2: 用于跨项目缺陷预测的更优隐私保护数据共享》, 发表于《2015 年 IEEE/ACM 第 37 届 IEEE 软件工程国际会议》第 1 卷。IEEE 出版社, 2015 年, 第 801 - 811 页。
- [26] G. 坎福拉、A. D. 卢西亚、M. D. 彭塔、R. 奥利韦托、A. 帕尼切拉和 S. 帕尼切拉, 《将缺陷预测作为一个多目标优化问题》, 《软件测试、验证与可靠性》, 第 25 卷, 第 4 期, 第 426 - 459 页, 2015 年。
- [27] 景 X.、吴 F.、董 X.、齐 F. 和徐 B., 《基于统一度量表示和基于典型相关分析的迁移学习的异构跨公司缺陷预测》, 载于《2015 年第 10 届软件工程基础联合会议论文集》, 2015 年, 第 496 - 507 页。
- [28] 何鹏、李博、刘晓、陈杰和马悦, 《基于简化度量集的软件缺陷预测实证研究》, 《信息与软件技术》, 2015 年第 59 卷, 第 170 - 190 页。
- [29] F. Zhang, Q. Zheng, Y. Zou 和 A. E. Hassan, “基于连通性的无监督分类器在跨项目缺陷预测中的应用”, 发表于 2016 年 IEEE/ACM 第 38 届软件工程国际会议 (ICSE)。IEEE, 2016 年, 第 309 - 320 页。
- [30] Y. 龟井、T. 福岛、S. 麦金托什、K. 山下、N. 宇林和 A. E. 哈桑, “使用跨项目模型研究即时缺陷预测”, 《实证软件工程》, 第 21 卷, 第 5 期, 第 2072 - 2106 页, 2016 年。
- [31] Y. 杨、Y. 周、J. 刘、Y. 赵、H. 陆、L. 徐、B. 徐和 H. 梁, 《工作量感知的即时缺陷预测: 简单的无监督模型可能优于有监督模型》, 发表于《2016 年第 24 届美国计算机协会软件工程基础国际研讨会论文集》, 2016 年, 第 157 - 168 页。
- [32] D. 柳、O. 崔和 J. 白克, 《基于支持向量机的价值认知增强在跨项目缺陷预测中的应用》, 《实证软件工程》, 第 21 卷, 第 1 期, 第 43 - 71 页, 2016 年。
- [33] 夏 X.、罗 D.、潘 S. J.、纳加潘 N.、王 X., 《九头蛇: 用于跨项目缺陷预测的大规模组合模型》, 《IEEE 软件工程汇刊》, 第 42 卷, 第 10 期, 第 977 - 998 页, 2016 年。
- [34] 景 X.-Y.、吴 F.、董 X. 和徐 B., 《一种改进的基于半监督判别式分析的缺陷预测框架, 用于解决项目内和跨项目的类别不平衡问题》, 《IEEE 软件工程汇刊》, 第 43 卷, 第 4 期, 第 321 - 339 页, 2016 年。
- [35] D. 柳和 J. 白克, 《面向跨项目缺陷预测的有效多目标朴素贝叶斯学习》, 《应用软计算》, 第 49 卷, 第 1062 - 1077 页, 2016 年。
- [36] F. 张、A. 莫克斯、I. 凯万卢和 Y. 邹, 《利用秩变换预测变量构建通用缺陷预测模型》, 《实证软件工程》, 第 21 卷, 第 5 期, 第 2107 - 2145 页, 2016 年。
- [37] R. 克里希纳、T. 门齐斯和 W. 傅, 《自动化过多? 领头羊效应及其对迁移学习的影响》, 载于《第 31 届 IEEE/ACM 国际自动化软件工程会议论文集》, 2016 年, 第 122 - 131 页。
- [38] 王 S.、刘 T.、谭 L., 《自动学习语义特征用于缺陷预测》, 发表于 2016 年 IEEE/ACM 第 38 届软件工程国际会议 (ICSE)。IEEE, 2016 年, 第 297 - 308 页。
- [39] D. 柳、J.-I. 张和 J. 白克, 《一种针对跨项目缺陷预测的迁移成本敏感提升方法》, 《软件质量期刊》, 第 25 卷, 第 1 期, 第 235 - 272 页, 2017 年。
- [40] J. 南、W. 傅、S. 金、T. 门齐斯和 L. 谭, 《异构缺陷预测》, 《IEEE 软件工程汇刊》, 第 44 卷, 第 9 期, 第 874 - 896 页, 2017 年。
- [41] C. 倪、W.-S. 刘、X. 陈、Q. 顾、D.-X. 陈和 Q.-G. 黄, “一种基于聚类的跨项目软件特征选择方法”

- 《软件缺陷预测》，《计算机科学与技术学报》，第 32 卷，第 6 期，第 1090 - 1107 页，2017 年。
- [42] 周扬，杨阳，卢华，陈亮，李阳，赵宇，钱军，徐博，《我们在旅程中取得了多大进展？跨项目缺陷预测的研究》，《ACM 软件工程与方法学汇刊》(TOSEM)，第 27 卷，第 1 期，第 1 - 51 页，2018 年。
- [43] X. 陈、Y. 赵、Q. 王和 Z. 袁，“Multi: 多目标工作量感知的即时软件缺陷预测”，《信息与软件技术》，第 93 卷，第 1 - 13 页，2018 年。
- [44] S. 侯赛尼、B. 图尔汗和 M. 曼蒂拉，《基于搜索的数据选择和特征选择对跨项目缺陷预测有效性的基准研究》，《信息与软件技术》，第 95 卷，第 296 - 312 页，2018 年。
- [45] R. 克里希纳和 T. 孟席斯，《领头羊：一种迁移学习的基线方法》，《IEEE 软件工程汇刊》，第 45 卷，第 11 期，第 1081 - 1105 页，2018 年。
- [46] Z. Li、X.-Y. Jing、F. Wu、X. Zhu、B. Xu 和 S. Ying，用于异构缺陷预测的成本敏感迁移核典型相关分析》，《自动化软件工程》，第 25 卷，第 2 期，第 201 - 245 页，2018 年。
- [47] 王 S.、刘 T.、南 J. 和谭 L.，《面向软件缺陷预测的深度语义特征学习》，《IEEE 软件工程汇刊》，第 46 卷，第 12 期，第 1267 - 1293 页，2018 年。
- [48] F. 吴、X.-Y. 景、Y. 孙、J. 孙、L. 黄、F. 崔和 Y. 孙，《跨项目和项目内半监督软件缺陷预测：一种统一方法》，《电气与电子工程师协会可靠性汇刊》，第 67 卷，第 2 期，第 581 - 597 页，2018 年。
- [49] H. K. 达姆、T. 范、S. W. 吴、T. 陈、J. 格伦迪、A. 戈斯、T. 金和 C.-J. 金，《一种基于深度树的软件缺陷预测模型》，arXiv 预印本 arXiv:1802.00921，2018 年。
- [50] Q. 黄、X. 夏和 D. 洛，《重新审视用于工作量感知的即时缺陷预测的监督和监督模型》，《实证软件工程》，第 24 卷，第 5 期，第 2823 - 2862 页，2019 年。
- [51] J. 陈、Y. 杨、K. 胡、Q. 宣、Y. 刘和 C. 杨，“用于软件缺陷预测的多视图迁移学习”，《IEEE 接入》，第 7 卷，第 8901 - 8916 页，2019 年。
- [52] X. 陈、D. 张、Y. 赵、Z. 崔和 C. 倪，《软件缺陷数量预测：无监督方法与有监督方法对比》，《信息与软件技术》，第 106 卷，第 161 - 181 页，2019 年。
- [53] 刘 C.、杨 D.、夏 X.、严 M 和张 X.，《一种用于跨项目缺陷预测的两阶段迁移学习模型》，《信息与软件技术》，第 107 卷，第 125 - 136 页，2019 年。
- [54] N. 斯里坎斯、S. 马宗德和 T. 孟席斯，《软件缺陷的早期生命周期预测：为何？如何？》，发表于 2021 年 IEEE/ACM 第 43 届软件工程国际会议 (ICSE)。美国加利福尼亚州洛杉矶阿拉米托斯：IEEE 计算机协会，2021 年 5 月，第 448 - 459 页。[在线]。获取地址：<https://doi.ieeecomputersociety.org/10.1109/ICSE43902.2021.00050>
- [55] B. 海尔彭和 P. 桑塔南，《软件调试、测试与验证》，《IBM 系统期刊》，第 41 卷，第 1 期，第 4 - 12 页，2002 年。F. 秋山，《软件系统调试示例》，载于《国际信息处理联合会大会 (1)》，第 71 卷，1971 年，第 353 - 359 页。
- [56] 万泽、夏鑫、A.E. 哈桑、罗迪、尹杰和杨鑫，“缺陷预测中的认知、期望与挑战”，《IEEE 软件工程汇刊》，2018 年。
- [57] T. J. 奥斯特兰德、E. J. 韦尤克和 R. M. 贝尔，《预测大型软件系统中故障的位置和数量》，《IEEE 软件工程汇刊》，第 31 卷，第 4 期，第 340 - 355 页，2005 年。
- [58] T. 门齐斯、J. 格林沃尔德和 A. 弗兰克，《通过数据挖掘静态代码属性学习缺陷预测器》，《IEEE 软件工程汇刊》，第 33 卷，第 1 期，第 2 - 13 页，2006 年。
- [59] A. T. 米西利、A. 贝内尔和 R. 卡莱，《基于人工智能的软件缺陷预测器：一项案例研究中的应用与益处》，《人工智能杂志》，第 32 卷，第 2 期，第 57 - 68 页，2011 年。
- [60] M. 金、D. 蔡和 S. 金，《对软件演化过程中 API 层面重构作用的实证研究》，收录于第 33 届软件工程国际会议论文集。美国计算机协会，2011 年，第 151 - 160 页。
- [61] F. 拉赫曼、S. 卡特里、E. T. 巴尔和 P. 德万布，《静态缺陷查找工具与统计预测方法的比较》，收录于《第 36 届软件工程国际会议论文集》，系列：ICSE 2014。美国纽约州纽约市：计算机协会，2014 年，第 424 - 434 页。[在线]。获取地址：<https://doi.org/10.1145/2568225.2568269>
- [62] L. C. 布赖恩德、W. L. 梅洛和 J. 武斯特，《面向对象软件项目中故障倾向模型的适用性评估》，《IEEE 软件工程汇刊》，第 28 卷，第 7 期，第 706 - 720 页，2002 年。
- [63] E. 科查古内利、T. 门齐斯和 E. 门德斯，《工作量估算中的迁移学习》，《实证软件工程》，第 20 卷，第 3 期，第 813 - 843 页，2015 年 6 月。[在线]。获取地址：<http://link.springer.com/10.1007/s10664-014-9300-5> [65] J. 南、S. J. 潘和 S. 金，《迁移缺陷学习》，收录于《软件工程国际会议论文集》，2013 年，第 382 - 391 页。
- [66] T. 齐默尔曼、N. 纳加潘、H. 高尔、E. 吉格和 B. 墨菲，《跨项目缺陷预测：关于数据、领域与流程的大规模实验》，载于《第 7 届欧洲软件工程会议与美国计算机协会软件基金会专题研讨会联合会议论文集》，2009 年，第 91 - 100 页。
- [67] E. J. 韦尤克、T. J. 奥斯特兰德和 R. M. 贝尔，《人多会坏事吗？利用开发人员数量改进缺陷预测模型》，《实证软件工程》，第 13 卷，第 5 期，第 539 - 559 页，2008 年。
- [68] A. 阿格拉瓦尔和 T. 孟席斯，《“更好的数据”比“更好的数据挖掘者”更好吗？》，发表于 2018 年 IEEE/ACM 第 40 届软件工程国际会议 (ICSE)。IEEE，2018 年，第 1050 - 1061 页。
- [69] G. 马修、A. 阿格拉瓦尔和 T. 孟席斯，《探寻软件工程研究的趋势》，《IEEE 软件工程汇刊》，2018 年。
- [70] Y. 龟井、E. 希哈布、B. 亚当斯、A. E. 哈桑、A. 莫克斯、A. 辛哈和 N. 宇林，《即时质量保证的大规模实证研究》，《IEEE 软件工程汇刊》，第 39 卷，第 6 期，第 757 - 773 页，2012 年。
- [71] C. 罗森、B. 格拉维、E. 施哈布，“提交大师：软件提交的分析与风险预测”，收录于《2015 年第 10 届软件工程基础联合会议论文集》。美国计算机协会，2015 年，第 966 - 969 页。[72] N. 穆纳亚、S. 克罗、C. 卡布里、M. 纳加潘，“为工程化软件项目整理 GitHub 资源”，《实证软件工程》，第 22 卷，第 6 期，第 3219 - 3253 页，2017 年。
- [73] 严敏、夏鑫、范阳、A.E. 哈桑、卢达威和李实，《即时缺陷识别与定位：一个两阶段框架》，《IEEE 软件工程汇刊》，2020 年。[74] N. 纳加潘和 T. 鲍尔，《使用相对代码变更量度量预测系统缺陷密度》，载于《第 27 届软件工程国际会议论文集》，美国计算机协会，2005 年，第 284 - 292 页。[75] 近藤真、D.M. 杰曼、水野修和崔恩熙，《上下文度量对即时缺陷预测的影响》，《实证软件工程》，第 25 卷，第 1 期，第 890 - 939 页，2020 年。[76] 夏鑫、E. 希哈布、龟井阳介、卢达威和王鑫，《预测移动应用的崩溃版本》，载于《第 10 届 ACM/IEEE 实证软件工程与测量国际研讨会论文集》，2016 年，第 1 - 10 页。
- [77] M. 近藤、C.-P. 贝泽默、Y. 龟井、A. E. 哈桑和 O. 水野，《特征约简技术对缺陷预测模型的影响》，《实证软件工程》，第 24 卷，第 4 期，第 1925 - 1963 页，2019 年。
- [78] E. 希哈布、Z. M. 江、W. M. 易卜拉欣、B. 亚当斯和 A. E. 哈桑，“理解代码和过程指标对发布后缺陷的影响：以 Eclipse 项目为例的研究”，载于《2010 年 ACM-IEEE 实证软件工程与测量国际研讨会论文集》，2010 年，第 1 - 10 页。
- [79] J. 斯利韦斯基、T. 齐默尔曼和 A. 泽勒，《变更何时引发修复？》，载于 2005 年挖掘软件仓库国际研讨会论文集，系列：MSR '05。美国纽约州纽约市：美国计算机协会，2005 年，第 1 - 5 页。[在线]。获取地址：<http://doi.acm.org/10.1145/1082983.1083147>
- [80] B. 戈特拉、S. 麦金托什和 A. E. 哈桑，“重新审视分类技术对缺陷预测模型性能的影响”，载于《第 37 届国际软件工程大会论文集第 1 卷》。IEEE 出版社，2015 年，第 789 - 800 页。
- [81] M. A. 霍尔和 G. 霍姆斯，《离散类数据挖掘中属性选择技术的基准测试》，《IEEE 知识与数据工程汇刊》，第 15 卷，第 6 期，第 1437 - 1447 页，2003 年。[82] N. V. 乔拉、K. W. 鲍耶、L. O. 霍尔和 W. P. 凯格尔迈耶，《SMOTE：合成少数类过采样技术》，《人工智能研究杂志》，第 16 卷，第 321 - 357 页，2002 年。[83] C. 坦蒂塔沃恩、S. 麦金托什、A. E. 哈桑和 K. 松本，《自动参数优化对……的影响》

- 《缺陷预测模型》，《IEEE 软件工程汇刊》，第 45 卷，第 7 期，第 683 - 711 页，2018 年。
- [84] 潘少杰、曾毅文、郭剑涛和杨强，《通过迁移成分分析进行领域自适应》，《IEEE 神经网络汇刊》，第 22 卷，第 2 期，第 199 - 210 页，2010 年。
- [85] J. 南、S. J. 潘和 S. 金，《迁移缺陷学习》，载于《软件工程国际会议论文集》，2013 年，第 382 - 391 页。
- [86] T. 门齐斯、B. 图尔汗、A. 贝内尔、G. 盖伊、B. 丘基奇和 Y. 江，《缺陷预测中的天花板效应的影响》，载于《第四届软件工程预测模型国际研讨会论文集》，2008 年，第 47 - 54 页。
- [87] 王 S. 和姚 X.，《利用类别不平衡学习进行软件缺陷预测》，《IEEE 可靠性汇刊》，第 62 卷，第 2 期，第 434 - 443 页，2013 年。
- [88] K. E. 贝宁、J. W. 基恩和 A. 蒙登，《关于软件缺陷预测中数据重采样方法的相对价值》，《实证软件工程》，第 24 卷，第 2 期，第 602 - 636 页，2019 年。
- [89] C. 坦蒂坦塔沃恩、A. E. 哈桑和 K. 松本，《类平衡技术对缺陷预测模型性能与解释的影响》，《IEEE 软件工程汇刊》，第 1 - 1 页，2018 年。
- [90] S. Yatish、J. Jiarpakdee、P. Thongtanunam 和 C. Tantithamthavorn，《挖掘软件缺陷：我们是否应该考虑受影响的版本？》，发表于 2019 年 IEEE/ACM 第 41 届软件工程国际会议 (ICSE)。IEEE，2019 年，第 654 - 665 页。
- [91] 闫 M、夏 X、罗 D、哈桑 A.E. 和李 S.，“回滚提交的特征描述与识别”，《实证软件工程》，2019 年，第 24 卷，第 4 期，第 2171 - 2208 页。
- [92] F. 张、A. E. 哈桑、S. 麦金托什和 Y. 邹，“使用求和来聚合软件指标会阻碍缺陷预测模型的性能”，《IEEE 软件工程汇刊》，第 43 卷，第 5 期，第 476 - 491 页，2016 年。
- [93] M. D' Ambrosio、M. Lanza 和 R. Robbes，《评估缺陷预测方法：一个基准和广泛比较》，《实证软件工程》，第 17 卷，第 4 - 5 期，第 531 - 577 页，2012 年。
- [94] F. 佩德雷戈萨、G. 瓦罗夸、A. 格拉姆福特、V. 米歇尔、B. 蒂里翁、O. 格里塞尔、M. 布朗德尔、P. 普雷滕霍费尔、R. 魏斯、V. 迪堡等，《Scikit-learn: Python 中的机器学习》，《机器学习研究杂志》，第 12 卷，第 2825 - 2830 页，2011 年。
- [95] C. 帕尔宁和 A. 奥尔索，《自动化调试技术真的在帮助程序员吗？》，发表于 2011 年软件测试与分析国际研讨会论文集。美国计算机协会，2011 年，第 199 - 209 页。
- [96] D. 陈、W. 傅、R. 克里希纳和 T. 门齐斯，《心理学在可行行动分析中的应用》，发表于 2018 年第 19 届软件工程基础研讨会 (FSE' 19)。
- [97] N. 米塔斯和 L. 安杰利斯，《通过多重比较算法对软件成本估算模型进行排序和聚类》，《IEEE 软件工程汇刊》，第 39 卷，第 4 期，第 537 - 551 页，2013 年 4 月。
- [98] A. 瓦尔加和 H. D. 德莱尼，《对麦格劳和王的 CL 通用语言效应量统计的批判与改进》，《教育与行为统计学杂志》，第 25 卷，第 2 期，第 101 - 132 页，2000 年。
- [99] N. 斯里坎斯和 T. 门齐斯，“评估从业者对软件缺陷预测的看法”，载于 2020 年 IEEE/ACM 第 42 届软件工程国际会议：实践中的软件工程 (ICSE - SEIP)。IEEE，2020 年，第 182 - 190 页。
- [100] A. 哈桑，“在伦敦大学学院克雷斯特公开研讨会上的演讲发言”，2017 年 3 月。
- [101] R. 索耶，“商业智能对分析和决策的影响取决于开发复杂度较低的应用程序”，收录于《商业智能研究的原理与应用》，IGI Global 出版社，2013 年，第 83 - 95 页。
- [102] M. Kim、T. Zimmermann、R. DeLine 和 A. Begel，“数据科学家在软件开发团队中日益凸显的作用”，载于《第 38 届软件工程国际会议论文集》，系列号 ICSE' 16。美国纽约州纽约市：美国计算机协会，2016 年，第 96 - 107 页。[在线]。获取地址：<http://doi.acm.org/10.1145/2884781.2884783>
- [103] S.-Y. Tan 和 T. Chan，“定义和概念化可操作洞察：以决策为中心的分析概念框架”，arXiv 预印本 arXiv:1606.03510，2016 年。
- [104] C. 伯德、T. 孟席斯和 T. 齐默尔曼，《分析软件数据的艺术与科学》，第 1 版。美国加利福尼亚州旧金山：摩根考夫曼出版社，2015 年。
- [105] A. 塞拉和 H. 本 - 加尔，《全球媒体公司、谷歌、脸书等员工离职的大数据分析》，2018 年 12 月，第 1 - 5 页。
- [106] C. 坦蒂坦塔沃恩、S. 麦金托什、A. E. 哈桑和 K. 松本，“缺陷预测模型分类技术的自动参数优化”，发表于《2016 年国际软件工程大会论文集》。美国计算机协会，2016 年，第 321 - 332 页。
- [107] W. 傅、T. 孟席斯和 X. 沈，“软件分析的调优：真的有必要吗？”，《信息与软件技术》，第 76 卷，第 135 - 146 页，2016 年。
- [108] A. 阿格拉瓦尔、W. 傅、D. 陈、X. 沈和 T. 门齐斯，《如何“规避”复杂的软件分析》，《IEEE 软件工程汇刊》，2019 年。
- [109] T. 黄、H. 庆·达姆、Y. 龟井、D. 洛和 N. 宇林，“Deepjit：一种用于即时缺陷预测的端到端深度学习框架”，载于 2019 年 IEEE/ACM 第 16 届挖掘软件仓库国际会议 (MSR)，2019 年，第 34 - 45 页。
- [110] M. 谭、L. 谭、S. 达拉和 C. 马约，《不平衡数据的在线缺陷预测》，载于 2015 年 IEEE/ACM 第 37 届 IEEE 软件工程国际会议，第 2 卷。IEEE，2015 年，第 99 - 108 页。